

LAPORAN PRAKTIKUM
STRUKTUR DATA
SORTING DAN GUI SORTING



Disusun oleh:

Ghinada Fathanawafa Algma
2511533008

Dosen Pengampu:
Dr. Wahyudi S.T. M.T

Asisten Praktikum:
Sherly Sukmadira

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa, berkat Rahmat dan karunia-Nya laporan praktikum Struktur Data tentang sorting dan GUI Sorting menggunakan bahasa Java ini dapat saya selesaikan dengan baik. Laporan ini saya susun sebagai bentuk pertanggungjawaban atas praktikum yang telah saya laksanakan.

Dalam penyusunan laporan ini saya banyak mendapat bimbingan serta dukungan dari beberapa pihak, oleh karena itu saya mengucapkan terima kasih kepada dosen pengampu, asisten praktikum dan rekan rekan praktikan yang turut membantu dalam pelaksanaan kegiatan praktikum.

Saya menyadari bahwa laporan praktikum ini jauh dari sempurna. Oleh karena itu kritik dan saran yang membangun sangat berguna untuk perkembangan lebih baik kedepannya. Semoga laporan praktikum ini dapat bermanfaat bagi saya serta pembaca, khususnya dalam memahami konsep Sorting.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	1
BAB II PEMBAHASAN	2
2.1 Dasar Materi.....	2
2.1.1 Insertion Sort.....	2
2.1.2 Selection Sort	2
2.1.3 Bubble Sort	3
2.2 Kode Program	3
2.2.1 InsertionSort_2511533008	3
2.2.2 SelectionSort_2511533008	5
2.2.3 BubbleSort_2511533008.....	7
2.2.4 InsertionGUI_2511533008	9
2.3 Output Program.....	15
2.3.1 Output InsertionSort_2511533008.....	15
2.3.2 Output SelectionSort_2511533008	15
2.3.3 Output BubbleSort_2511533008	15
2.3.4 Output InsertionGUI_2511533008	16
BAB III PENUTUP	17

3.1 Kesimpulan	17
DAFTAR PUSTAKA.....	18
TUGAS	19
1. Kode Program	19
2. Output Kode Program	23

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia pemrograman dan pengolahan data, algoritma sorting memiliki peran yang sangat penting karena memungkinkan data disusun secara teratur sehingga lebih mudah diakses, dianalisis, dan diproses. Sorting tidak hanya digunakan untuk mengurutkan data dalam urutan naik (ascending) atau turun (descending), tetapi juga menjadi dasar bagi berbagai algoritma lain seperti pencarian dan analisis statistik. Melalui praktikum struktur data ini, mahasiswa diharapkan dapat memahami konsep dasar pengurutan, mengenal berbagai jenis algoritma seperti Bubble Sort, Selection Sort, Insertion Sort, Merge Sort, Shell Sort dan Quick Sort, serta mampu menganalisis efisiensi masing-masing algoritma melalui kompleksitas waktu. Dengan adanya praktikum ini, kita tidak hanya belajar mengimplementasikan algoritma sorting dalam bahasa pemrograman, tetapi juga membandingkan performa algoritma terhadap berbagai ukuran data sehingga dapat menentukan metode yang paling sesuai untuk kebutuhan sistem.

1.2 Tujuan

1. Memahami konsep dasar algoritma sorting.
2. Dapat mengimplementasikan algoritma sorting dan membuat GUI nya.

1.3 Manfaat

1. Mengetahui dan memahami konsep dasar algoritma sorting.
2. Bisa membuat program algoritma sorting dengan GUI.

BAB II

PEMBAHASAN

2.1 Dasar Materi

2.1.1 Insertion Sort

Metode penyisipan (Insertion sort) bertujuan untuk menjadikan bagian sisi kiri array terurutkan sampai dengan seluruh array berhasil diurutkan. Metode ini mengurutkan bilangan-bilangan yang telah dibaca dan berikutnya secara berulang akan menyisipkan bilangan-bilangan dalam array yang belum terbaca ke sisi kiri array yang telah terurut.

Running time bukan hanya bergantung pada ukuran array, namun juga pada susunan isinya.

BEST CASE:

- Array sudah dalam keadaan terurut naik
- Loop terdalam tidak pernah dieksekusi
- Jumlah pergeseran : $2(n-1)$
- Jumlah pembandingan key (C) : $(n-1)$

WORST CASE:

- Array dalam urutan kebalikannya
- Loop terdalam dieksekusi sebanyak $p-1$ kali, untuk $p = 2, 3, \dots, n$
- Jumlah pergeseran : $2(n-1) + (1 + 2 + \dots + n-1) = 2(n-1) + n * (n-1) / 2$
- Jumlah pembandingan key : $(1 + 2 + \dots + n-1) = n * (n-1) / 2$

2.1.2 Selection Sort

Cek seluruh elemen array, temukan nilai terkecil dan tukarkan posisinya dengan posisi nilai yang tersimpan pada posisi pertama dari array.

Secara umum, yang dilakukan dalam metode seleksi adalah pembandingan key (elemen pada posisi min) serta penukaran elemen. Sehingga untuk menganalisis

metode ini harus dihitung jumlah pembandingannya serta jumlah penukaran elemennya.

Dalam metode ini, jumlah pembandingan untuk best case & worst casenya sama. Memindahkan dari kanan ke kiri, meletakkan elemen ke posisi finalnya tanpa merevisi lagi posisi tsb. Menghabiskan sebagian besar waktu untuk mencari elemen terkecil pada sisi array yang belum terurut.

2.1.3 Bubble Sort

Metode gelembung (bubble sort) disebut dengan metode penukaran (exchange sort) adalah metode yang mengurutkan data dengan cara membandingkan masing-masing elemen, kemudian melakukan penukaran bila perlu. Metode ini mudah dipahami > diprogram, tetapi bila dibandingkan dengan metode lain yang kita pelajari, metode ini merupakan metode yang paling tidak efisien.

BEST CASE:

- Array sudah dalam keadaan terurut naik
- Jumlah pembandingan key (C) : $n-1$
- Jumlah swap = 0
- Jumlah pergeseran (M) : 0

WORST CASE:

- Array dalam urutan kebalikannya
- Jumlah pembandingan key (C) : $n-1$
- Jumlah swap = $(n-1) + (n-2) + \dots + 1 = n * (n-1) / 2$
- Jumlah pergeseran (M) : $3 * n * (n-1) / 2$

2.2 Kode Program

2.2.1 InsertionSort_2511533008

Kode Program:

```
package pekan7_2511533008;
```

```

public class InsertionSort_2511533008 {
    public static void insertionSort_2511533008 (int[] arr_3008) {
        int n_3008 = arr_3008.length;
        for (int i_3008 = 1; i_3008 < n_3008; i_3008++) {
            int key_3008 = arr_3008[i_3008];
            int j_3008 = i_3008 - 1;
            while (j_3008 >= 0 && arr_3008[j_3008] >
key_3008) {
                arr_3008[j_3008 + 1] = arr_3008[j_3008];
                j_3008--;
            }
            arr_3008[j_3008 + 1] = key_3008;
        }
    }

    public static void main(String[] args) {
        int arr_3008[] = {23, 78, 45, 8, 32, 56, 1};
        int n_3008 = arr_3008.length;
        System.out.printf("array yang belum terurut:\n");
        for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
            System.out.print(arr_3008[i_3008]+ " ");
        System.out.println("");
        insertionSort_2511533008(arr_3008);
        System.out.printf("array yang sudah terurut:\n");
        for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
            System.out.print(arr_3008[i_3008]+ " ");
        System.out.println("");
    }
}

```

Penjelasan:

1. Fungsi Insertion

```

public class InsertionSort_2511533008 {
    public static void insertionSort_2511533008 (int[] arr_3008) {
        int n_3008 = arr_3008.length;
        for (int i_3008 = 1; i_3008 < n_3008; i_3008++) {
            int key_3008 = arr_3008[i_3008];
            int j_3008 = i_3008 - 1;
            while (j_3008 >= 0 && arr_3008[j_3008] > key_3008) {
                arr_3008[j_3008 + 1] = arr_3008[j_3008];
                j_3008--;
            }
            arr_3008[j_3008 + 1] = key_3008;
        }
    }
}

```

Gambar 2.2.1.1 fungsi insertion

- `n_3008 = arr_3008.length` → Menyimpan panjang array.

- `for (int i_3008 = 1; i_3008 < n_3008; i_3008++)` → Loop dari elemen kedua hingga akhir.
- `key_3008 = arr_3008[i_3008]` → Menyimpan elemen yang akan dibandingkan.
- `j_3008 = i_3008 - 1` → Pointer untuk elemen sebelumnya.
- `while (j_3008 >= 0 && arr_3008[j_3008] > key_3008)` → Selama elemen sebelumnya lebih besar dari key, geser elemen ke kanan.
- `arr_3008[j_3008 + 1] = key_3008` → Menempatkan key di posisi yang tepat.

2. Fungsi main

```
public static void main(String[] args) {
    int arr_3008[] = {23, 78, 45, 8, 32, 56, 1};
    int n_3008 = arr_3008.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
    insertionSort_2511533008(arr_3008);
    System.out.printf("array yang sudah terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
}
```

Gambar 2.2.1.2 fungsi utama

Membuat array `arr_3008` berisi angka acak. Menampilkan isi array sebelum diurutkan. Memanggil fungsi `insertionSort_2511533008` untuk mengurutkan array. Menampilkan isi array setelah diurutkan.

2.2.2 SelectionSort_2511533008

Kode Program:

```
package pekan7_2511533008;

public class SelectionSort_2511533008 {
    public static void selectionSort_2511533008 (int[] arr_3008) {
        int n_3008 = arr_3008.length;
        for (int i_3008 = 0; i_3008 < n_3008; i_3008++) {
            int minIndex_3008 = i_3008;
            for (int j_3008 = i_3008 + 1; j_3008 < n_3008;
j_3008++ ) {
```

```

        if (arr_3008[j_3008] <
arr_3008[minIndex_3008]) {
            minIndex_3008 = j_3008;
        }
    }
    int temp_3008 = arr_3008[i_3008];
    arr_3008[i_3008] = arr_3008[minIndex_3008];
    arr_3008[minIndex_3008] = temp_3008;
}
}

public static void main(String[] args) {
    int arr_3008[] = {23, 78, 45, 8, 32, 56, 1};
    int n_3008 = arr_3008.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
    selectionSort_2511533008(arr_3008);
    System.out.printf("array yang sudah terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
}
}

```

Penjelasan:

1. Fungsi selection

```

public class SelectionSort_2511533008 {
    public static void selectionSort_2511533008 (int[] arr_3008) {
        int n_3008 = arr_3008.length;
        for (int i_3008 = 0; i_3008 < n_3008; i_3008++) {
            int minIndex_3008 = i_3008;
            for (int j_3008 = i_3008 + 1; j_3008 < n_3008; j_3008++ ) {
                if (arr_3008[j_3008] < arr_3008[minIndex_3008]) {
                    minIndex_3008 = j_3008;
                }
            }
            int temp_3008 = arr_3008[i_3008];
            arr_3008[i_3008] = arr_3008[minIndex_3008];
            arr_3008[minIndex_3008] = temp_3008;
        }
    }
}

```

Gambar 2.2.2.1 fungsi selection

- n_3008 = arr_3008.length → Panjang array.

- for (int i_3008 = 0; i_3008 < n_3008; i_3008++) → Loop dari elemen pertama hingga terakhir.
- minIndex_3008 = i_3008 → Asumsikan elemen saat ini adalah yang terkecil.
- for (int j_3008 = i_3008 + 1; j_3008 < n_3008; j_3008++) → Cari elemen terkecil di sisa array.
- if (arr_3008[j_3008] < arr_3008[minIndex_3008]) → Jika ditemukan elemen lebih kecil, simpan indeksny.
- Setelah loop dalam selesai, **tukar** elemen terkecil dengan elemen di posisi i_3008.

2. Fungsi main

```
public static void main(String[] args) {
    int arr_3008[] = {23, 78, 45, 8, 32, 56, 1};
    int n_3008 = arr_3008.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
    selectionSort_2511533008(arr_3008);
    System.out.printf("array yang sudah terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
}
```

Gambar 2.2.2.2 fungsi utama

Membuat array arr_3008 berisi angka acak. Menampilkan isi array sebelum diurutkan. Memanggil fungsi selectionSort_2511533008 untuk mengurutkan array. Menampilkan isi array setelah diurutkan.

2.2.3 BubbleSort_2511533008

Kode Program:

```
package pekan7_2511533008;

public class BubleSort_2511533008 {
    public static void bubbleSort_2511533008(int[] arr_3008) {
        int n_3008 = arr_3008.length;
        for (int i_3008 = 0; i_3008 < n_3008; i_3008++) {
            for (int j_3008 = 0; j_3008 < n_3008 - i_3008 - 1; j_3008++) {
```

```

        if (arr_3008[j_3008] > arr_3008[j_3008 +
1]) {
            int temp_3008 = arr_3008[j_3008];
            arr_3008[j_3008] = arr_3008[j_3008
+ 1];
            arr_3008[j_3008 + 1] = temp_3008;
            //System.out.println("data:" +
arr[j_3008] + " " + arr_3008[j_3008 + 1]);
        }
    }
}

public static void main(String[] args) {
    int arr_3008[] = {23, 78, 45, 8, 32, 56, 1};
    int n_3008 = arr_3008.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
    bubbleSort_2511533008(arr_3008);
    System.out.printf("array yang sudah terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
}
}

```

Penjelasan:

1. Fungsi Bubble

```

public class BubbleSort_2511533008 {
    public static void bubbleSort_2511533008(int[] arr_3008) {
        int n_3008 = arr_3008.length;
        for (int i_3008 = 0; i_3008 < n_3008; i_3008++) {
            for (int j_3008 = 0; j_3008 < n_3008 - i_3008 - 1; j_3008++) {
                if (arr_3008[j_3008] > arr_3008[j_3008 + 1]) {
                    int temp_3008 = arr_3008[j_3008];
                    arr_3008[j_3008] = arr_3008[j_3008 + 1];
                    arr_3008[j_3008 + 1] = temp_3008;
                    //System.out.println("data:" + arr[j_3008] + " " + arr_3008[j_3008 + 1]);
                }
            }
        }
    }
}

```

Gambar 2.2.3.1 fungsi bubble

- `n_3008 = arr_3008.length` → Panjang array.
- `for (int i_3008 = 0; i_3008 < n_3008; i_3008++)` → Loop sebanyak panjang array.

- `for (int j_3008 = 0; j_3008 < n_3008 - i_3008 - 1; j_3008++)` → Loop dalam untuk membandingkan elemen bersebelahan.
- `if (arr_3008[j_3008] > arr_3008[j_3008 + 1])` → Jika elemen kiri lebih besar dari elemen kanan, tukar posisinya.
- Proses ini dilakukan berulang sehingga elemen terbesar “menggelembung” ke posisi akhir array.

2. Fungsi main

```
public static void main(String[] args) {
    int arr_3008[] = {23, 78, 45, 8, 32, 56, 1};
    int n_3008 = arr_3008.length;
    System.out.printf("array yang belum terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
    bubbleSort_2511533008(arr_3008);
    System.out.printf("array yang sudah terurut:\n");
    for (int i_3008 = 0; i_3008 < n_3008; i_3008++)
        System.out.print(arr_3008[i_3008]+ " ");
    System.out.println("");
}
```

Gambar 2.2.3.2 fungsi utama

Membuat array `arr_3008` berisi angka acak. Menampilkan isi array sebelum diurutkan. Memanggil fungsi `bubbleSort_2511533008` untuk mengurutkan array. Menampilkan isi array setelah diurutkan.

2.2.4 InsertionGUI_2511533008

Kode Program:

```
package pekan7_2511533008;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.Color;
import java.awt.Dimension;

import javax.swing.JFrame;
import javax.swing.JPanel;
import javax.swing.JLabel;
import javax.swing.JButton;
import javax.swing.JTextField;
import javax.swing.JTextArea;
import javax.swing.JScrollPane;
```

```

import javax.swing.JOptionPane;
import javax.swing.BorderFactory;
import javax.swing.SwingConstants;

public class InsertionSortGUI_2511533008 extends JFrame {

    private static final long serialVersionUID = 1L;

    private int[] array_3008;
    private JLabel[] labelArray_3008;
    private JButton stepButton_3008, resetButton_3008,
setButton_3008;
    private JTextField inputField_3008;
    private JPanel panelArray_3008;
    private JTextArea stepArea_3008;

    private int i_3008 = 1, j_3008;
    private boolean sorting_3008 = false;
    private int stepCount_3008 = 1;

    public InsertionSortGUI_2511533008() {
        setTitle("Insertion Sort: Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_3008 = new JPanel(new FlowLayout());
        inputField_3008 = new JTextField(30);
        setButton_3008 = new JButton("Set Array");
        inputPanel_3008.add(new JLabel("Masukkan angka (pisahkan
dengan koma:"));
        inputPanel_3008.add(inputField_3008);
        inputPanel_3008.add(setButton_3008);

        // Panel array visual
        panelArray_3008 = new JPanel();
        panelArray_3008.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_3008 = new JPanel();
        stepButton_3008 = new JButton("Langkah Selanjutnya");
        resetButton_3008 = new JButton("Reset");
        stepButton_3008.setEnabled(false);
        controlPanel_3008.add(stepButton_3008);
        controlPanel_3008.add(resetButton_3008);

        // Area teks untuk log langkah-langkah
        stepArea_3008 = new JTextArea(8, 60);
        stepArea_3008.setEditable(false);
        stepArea_3008.setFont(new Font("Monospaced", Font.PLAIN,
14));

```

```

        JScrollPane scrollPane_3008 = new
JScrollPane(stepArea_3008);

        // Tambahkan panel ke frame
add(inputPanel_3008, BorderLayout.NORTH);
add(panelArray_3008, BorderLayout.CENTER);
add(controlPanel_3008, BorderLayout.SOUTH);
add(scrollPane_3008, BorderLayout.EAST);

        // Event Set Array
setButton_3008.addActionListener(e ->
setArrayFromInput_3008());

        // Event Langkah Selanjutnya
stepButton_3008.addActionListener(e -> performStep_3008());

        // Event Reset
resetButton_3008.addActionListener(e -> reset_3008());
    }

    private void setArrayFromInput_3008() {
        String text_3008 = inputField_3008.getText().trim();

        if (text_3008.isEmpty()) return;

        String[] parts_3008 = text_3008.split(",");
        array_3008 = new int[parts_3008.length];

        try {
            for (int k_3008 = 0; k_3008 < parts_3008.length;
k_3008++) {
                array_3008[k_3008] =
Integer.parseInt(parts_3008[k_3008].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(
                this,
                "Masukkan hanya angka yang dipisahkan dengan
koma",
                "Error",
                JOptionPane.ERROR_MESSAGE
            );
            return;
        }

        i_3008 = 0;
        stepCount_3008 = 1;
        sorting_3008 = true;

        stepButton_3008.setEnabled(true);
        stepArea_3008.setText("");

        panelArray_3008.removeAll();
    }

```

```

        labelArray_3008 = new JLabel[array_3008.length];

        for (int k_3008 = 0; k_3008 < array_3008.length; k_3008++) {
            labelArray_3008[k_3008] = new
JLabel(String.valueOf(array_3008[k_3008]));
            labelArray_3008[k_3008].setFont(new Font("Arial",
Font.BOLD, 24));

labelArray_3008[k_3008].setBorder(BorderFactory.createLineBorder(Col
or.BLACK));
            labelArray_3008[k_3008].setPreferredSize(new
Dimension(50, 50));

labelArray_3008[k_3008].setHorizontalAlignment(SwingConstants.CENTER
);
            panelArray_3008.add(labelArray_3008[k_3008]);
        }
        panelArray_3008.revalidate();
        panelArray_3008.repaint();
    }
    private void performStep_3008() {
        if (i_3008 < array_3008.length && sorting_3008) {
            int key_3008 = array_3008[i_3008];
            j_3008 = i_3008 - 1;
            StringBuilder stepLog_3008 = new StringBuilder();
            stepLog_3008.append("Langkah
").append(stepCount_3008).append(": Masukkan
").append(key_3008).append("\n");
            while (j_3008 >= 0 && array_3008[j_3008] > key_3008) {
                array_3008[j_3008 + 1] = array_3008[j_3008]; j_3008-
-;
            }
            array_3008[j_3008 + 1] = key_3008;
            updateLabels_3008();
            stepLog_3008.append("Hasil:
").append(arrayToString_3008(array_3008)).append("\n\n");
            stepArea_3008.append(stepLog_3008.toString());
            i_3008++;
            stepCount_3008++;
        }
        if (i_3008 == array_3008.length) {
            sorting_3008 = false;
            stepButton_3008.setEnabled(false);
            JOptionPane.showMessageDialog(this, "Sorting selesai!");
        }
    }
    private void updateLabels_3008() {
        for (int k_3008 = 0; k_3008 < array_3008.length; k_3008++) {
labelArray_3008[k_3008].setText(String.valueOf(array_3008[k_3008]));
        }
    }
}

```



```

private void reset_3008() {
    inputField_3008.setText("");
    panelArray_3008.removeAll();
    panelArray_3008.revalidate();
    panelArray_3008.repaint();
    stepArea_3008.setText("");
    stepButton_3008.setEnabled(false);
    sorting_3008 = false;
    i_3008 = 1;
    stepCount_3008 = 1;
}
private String arrayToString_3008(int[] arr_3008) {
    StringBuilder sb_3008 = new StringBuilder();
    for (int k_3008 = 0; k_3008 < arr_3008.length; k_3008++) {
        sb_3008.append(arr_3008[k_3008]);
        if (k_3008 < arr_3008.length - 1) {
            sb_3008.append(", ");
        }
    }
    return sb_3008.toString();
}
public static void main(String[] args) {
    javax.swing.SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511533008 gui_3008 =
            new InsertionSortGUI_2511533008();
        gui_3008.setVisible(true);
    });
}
}

```

Penejelasan:

Class InsertionSortGUI_2511533008 → Turunan dari JFrame, berfungsi sebagai jendela utama aplikasi.

Komponen utama:

- **Input field** (JTextField) → tempat memasukkan array angka.
- **Set Array button** → untuk memproses input menjadi array.
- **Step button** → mengeksekusi satu langkah *Insertion Sort*.
- **Reset button** → menghapus semua data dan mengulang dari awal.
- **Panel array** → menampilkan angka dalam bentuk label.
- **Text area log** → menampilkan catatan langkah-langkah sorting.

Alur program berjalan:

1. Input Array

- User memasukkan angka dipisahkan koma.
- Program memvalidasi input (hanya angka).
- Array ditampilkan sebagai label di panel.

2. Proses Sorting Langkah demi Langkah

- Tombol *Langkah Selanjutnya* akan menjalankan satu iterasi *Insertion Sort*.
- Elemen yang sedang diproses (key) dibandingkan dengan elemen sebelumnya.
- Jika lebih kecil, elemen digeser ke kanan.
- Setelah selesai, array diperbarui di panel dan log langkah ditambahkan di stepArea.

3. Reset

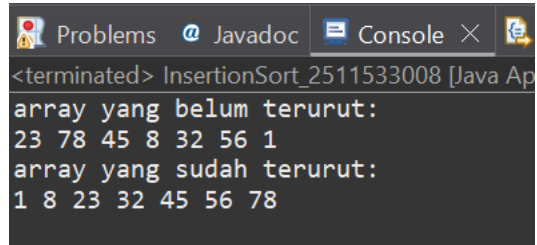
- Menghapus input, panel array, dan log.
- Mengembalikan status ke awal.

Bagian bagian yang penting pada program:

- **setArrayFromInput_3008()** → Membaca input, memvalidasi, lalu menampilkan array sebagai label.
- **performStep_3008()** → Menjalankan satu langkah *Insertion Sort* dan memperbarui tampilan.
- **updateLabels_3008()** → Mengubah teks label sesuai isi array terbaru.
- **reset_3008()** → Membersihkan semua komponen agar siap digunakan lagi.
- **arrayToString_3008()** → Mengubah array menjadi string untuk ditampilkan di log.

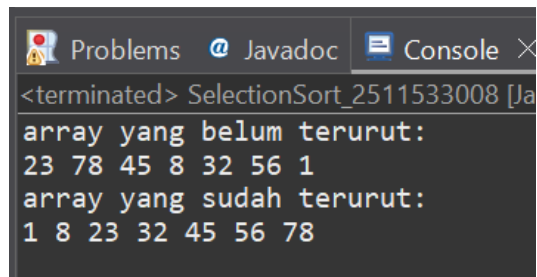
2.3 Output Program

2.3.1 Output InsertionSort_2511533008



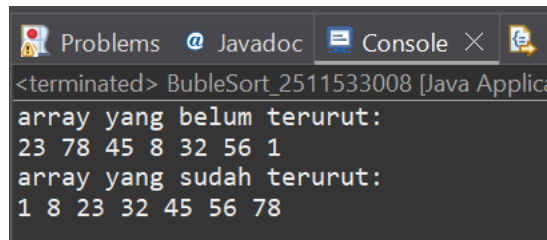
```
<terminated> InsertionSort_2511533008 [Java Ap  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang sudah terurut:  
1 8 23 32 45 56 78
```

2.3.2 Output SelectionSort_2511533008



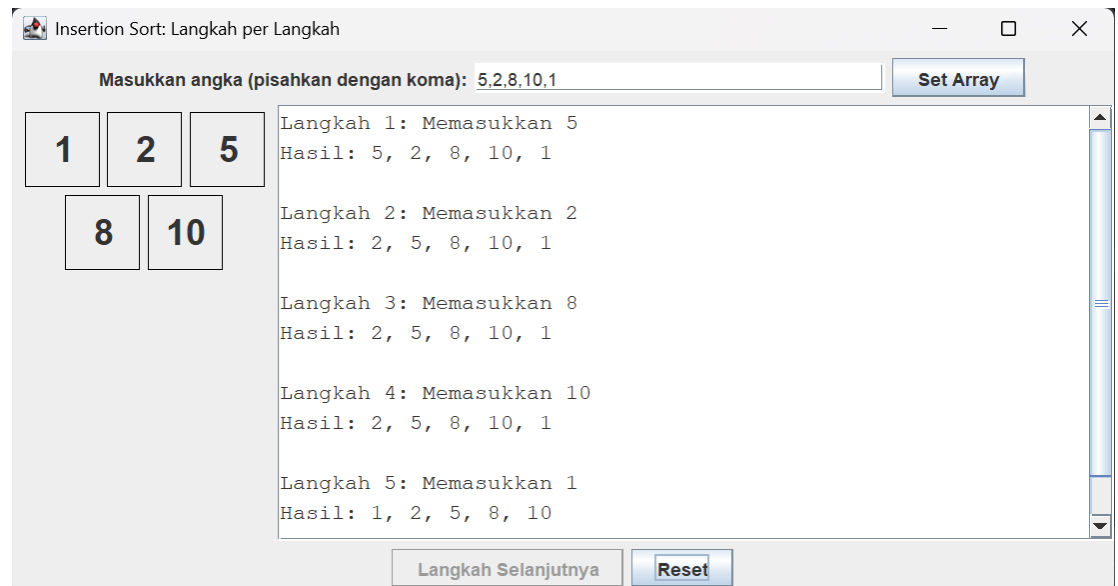
```
<terminated> SelectionSort_2511533008 [Ja  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang sudah terurut:  
1 8 23 32 45 56 78
```

2.3.3 Output BubbleSort_2511533008



```
<terminated> BubleSort_2511533008 [Java Applic  
array yang belum terurut:  
23 78 45 8 32 56 1  
array yang sudah terurut:  
1 8 23 32 45 56 78
```

2.3.4 Output InsertionGUI_2511533008



BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa algoritma sorting merupakan salah satu konsep fundamental dalam struktur data yang berfungsi untuk mengurutkan elemen-elemen data agar lebih terstruktur dan mudah diakses. Melalui implementasi algoritma seperti Bubble Sort, Selection Sort, dan Insertion Sort, kita dapat memahami perbedaan cara kerja masing-masing algoritma serta menganalisis efisiensi berdasarkan kompleksitas waktu. Praktikum ini juga menunjukkan bahwa pemilihan algoritma yang tepat sangat bergantung pada ukuran dan kondisi data yang diolah. Dengan menguasai konsep sorting, kita memperoleh bekal penting untuk mengembangkan algoritma lain yang lebih kompleks, meningkatkan efisiensi sistem, serta menerapkan pengurutan data dalam berbagai aplikasi nyata seperti database, sistem informasi, dan analisis statistik.

DAFTAR PUSTAKA

- [1] Universitas Andalas, "Pertemuan 10: Sorting Algorithm," Modul Struktur Data, 2026.
- [2] Universitas Andalas, "Pertemuan 11: Sorting Algorithm Bubble and Shell," Modul Struktur Data, 2026.
- [3] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser, *Data Structures and Algorithms in Java*, 6th ed. Wiley, 2014.
- [4] I. Lim, *Diktat Struktur Data*. Bandung: Institut Teknologi Bandung, 2008.

TUGAS

1. Kode Program

```
package pekan7_2511533008;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.util.ArrayList;

class Mahasiswa_3008 {
    private String nama_3008;
    private String nim_3008;
    private String prodi_3008;

    public Mahasiswa_3008(String nama_3008, String nim_3008, String
prodi_3008) {
        this.nama_3008 = nama_3008;
        this.nim_3008 = nim_3008;
        this.prodi_3008 = prodi_3008;
    }

    public String getNama_3008() { return nama_3008; }
    public String getNim_3008() { return nim_3008; }
    public String getProdi_3008() { return prodi_3008; }

    @Override
    public String toString() {
        return nama_3008 + " - " + nim_3008 + " - " + prodi_3008;
    }
}

public class MahasiswaGUI_2511533008 extends JFrame {
    private JTextField tfNama_3008, tfNim_3008, tfProdi_3008;
    private JButton btnTambah_3008, btnHapus_3008, btnSort_3008;
    private JComboBox<String> cbAlgoritma_3008;
    private JTable tableAwal_3008, tableAkhir_3008;
    private DefaultTableModel modelAwal_3008, modelAkhir_3008;
    private JTextArea areaProses_3008;

    private ArrayList<Mahasiswa_3008> listMahasiswa_3008 = new
ArrayList<>();

    public MahasiswaGUI_2511533008() {
        setTitle("Sorting Mahasiswa");
        setSize(900, 500);
        setDefaultCloseOperation(EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        // Panel Input
        JPanel panelInput_3008 = new JPanel(new GridLayout(4, 2));
```

```

tfNama_3008 = new JTextField();
tfNim_3008 = new JTextField();
tfProdi_3008 = new JTextField();
btnTambah_3008 = new JButton("Tambah");
btnHapus_3008 = new JButton("Hapus");
panelInput_3008.add(new JLabel("Nama:"));
panelInput_3008.add(tfNama_3008);
panelInput_3008.add(new JLabel("NIM:"));
panelInput_3008.add(tfNim_3008);
panelInput_3008.add(new JLabel("Prodi:"));
panelInput_3008.add(tfProdi_3008);
panelInput_3008.add(btnTambah_3008);
panelInput_3008.add(btnHapus_3008);

// Tabel sebelum dan sesudah
modelAwal_3008 = new DefaultTableModel(new String[]{"Nama", "NIM",
"Prodi"}, 0);
tableAwal_3008 = new JTable(modelAwal_3008);

modelAkhir_3008 = new DefaultTableModel(new String[]{"Nama", "NIM",
"Prodi"}, 0);
tableAkhir_3008 = new JTable(modelAkhir_3008);

JPanel panelTables_3008 = new JPanel(new GridLayout(1, 2));
panelTables_3008.add(new JScrollPane(tableAwal_3008));
panelTables_3008.add(new JScrollPane(tableAkhir_3008));

// Panel Sorting
JPanel panelSort_3008 = new JPanel();
cbAlgoritma_3008 = new JComboBox<>(new String[]{"Insertion Sort",
"Selection Sort", "Bubble Sort"});
btnSort_3008 = new JButton("Mulai Sorting");
panelSort_3008.add(cbAlgoritma_3008);
panelSort_3008.add(btnSort_3008);

// Area Proses
areaProses_3008 = new JTextArea(10, 50);
areaProses_3008.setEditable(false);

add(panelInput_3008, BorderLayout.NORTH);
add(panelTables_3008, BorderLayout.CENTER);
add(panelSort_3008, BorderLayout.SOUTH);
add(new JScrollPane(areaProses_3008), BorderLayout.EAST);

// Action Listener
btnTambah_3008.addActionListener(e -> {
    String nama_3008 = tfNama_3008.getText();
    String nim_3008 = tfNim_3008.getText();
    String prodi_3008 = tfProdi_3008.getText();
    if (!nama_3008.isEmpty() && !nim_3008.isEmpty() &&
!prodi_3008.isEmpty()) {
        Mahasiswa_3008 m_3008 = new Mahasiswa_3008(nama_3008,
nim_3008, prodi_3008);

```



```

        listMahasiswa_3008.add(m_3008);
        modelAwal_3008.addRow(new Object[]{nama_3008, nim_3008,
prodi_3008});

        tfNama_3008.setText("");
        tfNim_3008.setText("");
        tfProdi_3008.setText("");
        tfNama_3008.requestFocus();
    }
});

btnHapus_3008.addActionListener(e -> {
    int row_3008 = tableAwal_3008.getSelectedRow();
    if (row_3008 >= 0) {
        listMahasiswa_3008.remove(row_3008);
        modelAwal_3008.removeRow(row_3008);
    }
});

btnSort_3008.addActionListener(e -> {
    String algo_3008 = (String) cbAlgoritma_3008.getSelectedItem();
    areaProses_3008.setText("");
    ArrayList<Mahasiswa_3008> tempList_3008 = new
ArrayList<>(listMahasiswa_3008);

    switch (algo_3008) {
        case "Insertion Sort": insertionSort_3008(tempList_3008);
break;
        case "Selection Sort": selectionSort_3008(tempList_3008);
break;
        case "Bubble Sort": bubbleSort_3008(tempList_3008); break;
    }
    refreshTableAkhir_3008(tempList_3008);
});
}

private void insertionSort_3008(ArrayList<Mahasiswa_3008> tempList) {
    for (int i_3008 = 1; i_3008 < tempList.size(); i_3008++) {
        Mahasiswa_3008 key_3008 = tempList.get(i_3008);
        int j_3008 = i_3008 - 1;
        while (j_3008 >= 0 && tempList.get(j_3008).getNama_3008()
            .compareToIgnoreCase(key_3008.getNama_3008()) > 0) {
            tempList.set(j_3008 + 1, tempList.get(j_3008));
            j_3008--;
        }
        tempList.set(j_3008 + 1, key_3008);

        areaProses_3008.append("Langkah " + i_3008 + ": ");
        for (Mahasiswa_3008 m_3008 : tempList) {
            areaProses_3008.append(m_3008.toString() + "; ");
        }
        areaProses_3008.append("\n");
    }
}

```

```

    }

    private void selectionSort_3008(ArrayList<Mahasiswa_3008> tempList) {
        for (int i_3008 = 0; i_3008 < tempList.size() - 1; i_3008++) {
            int minIdx_3008 = i_3008;
            for (int j_3008 = i_3008 + 1; j_3008 < tempList.size();
j_3008++) {
                if (tempList.get(j_3008).getNama_3008()
.compareToIgnoreCase(tempList.get(minIdx_3008).getNama_3008()) < 0) {
                    minIdx_3008 = j_3008;
                }
            }
            Mahasiswa_3008 temp_3008 = tempList.get(minIdx_3008);
            tempList.set(minIdx_3008, tempList.get(i_3008));
            tempList.set(i_3008, temp_3008);

            areaProses_3008.append("Pass " + (i_3008 + 1) + ": ");
            for (Mahasiswa_3008 m_3008 : tempList) {
                areaProses_3008.append(m_3008.toString() + "; ");
            }
            areaProses_3008.append("\n");
        }
    }

    private void bubbleSort_3008(ArrayList<Mahasiswa_3008> tempList) {
        for (int i_3008 = 0; i_3008 < tempList.size() - 1; i_3008++) {
            for (int j_3008 = 0; j_3008 < tempList.size() - i_3008 - 1;
j_3008++) {
                if (tempList.get(j_3008).getNama_3008()
.compareToIgnoreCase(tempList.get(j_3008 +
1).getNama_3008()) > 0) {
                    Mahasiswa_3008 temp_3008 = tempList.get(j_3008);
                    tempList.set(j_3008, tempList.get(j_3008 + 1));
                    tempList.set(j_3008 + 1, temp_3008);
                }
            }
            areaProses_3008.append("Pass " + (i_3008 + 1) + ": ");
            for (Mahasiswa_3008 m_3008 : tempList) {
                areaProses_3008.append(m_3008.toString() + "; ");
            }
            areaProses_3008.append("\n");
        }
    }

    private void refreshTableAkhir_3008(ArrayList<Mahasiswa_3008> tempList)
    {
        modelAkhir_3008.setRowCount(0); // kosongkan tabel hasil
        for (Mahasiswa_3008 m_3008 : tempList) {
            modelAkhir_3008.addRow(new Object[]{
                m_3008.getNama_3008(),
                m_3008.getNim_3008(),
                m_3008.getProdi_3008()
            });
        }
    }

```

```

    });
}
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> new
MahasiswaGUI_2511533008().setVisible(true));
}
}

```

2. Output Kode Program

Sorting Mahasiswa

Nama:

NIM:

Prodi:

Tambah			Hapus		
Nama	NIM	Prodi	Nama	NIM	Prodi
Ghinada	2511533008	IF	Annisa	2511532024	IF
Kinaya	2511533008	IF	Ghinada	2511533008	IF
Annisa	2511532024	IF	Haliya	2511532002	IF
Syasya	2511531018	IF	Kinaya	2511533008	IF
Haliya	2511532002	IF	Syasya	2511531018	IF

Langkah 1: Ghinada - 2511533008 - IF; Kinaya - 2511533008 - IF; Annisa - 2511532024 - IF; Haliya - 2511532002 - IF; Syasya - 2511531018 - IF

Langkah 2: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Kinaya - 2511533008 - IF; Haliya - 2511532002 - IF; Syasya - 2511531018 - IF

Langkah 3: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Kinaya - 2511533008 - IF; Haliya - 2511532002 - IF; Syasya - 2511531018 - IF

Langkah 4: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Haliya - 2511532002 - IF; Syasya - 2511531018 - IF; Ghinada - 2511533008 - IF

Insertion Sort

Gambar 1 output Insertion sort

Sorting Mahasiswa

Nama:

NIM:

Prodi:

Tambah			Hapus		
Nama	NIM	Prodi	Nama	NIM	Prodi
Ghinada	2511533008	IF	Annisa	2511532024	IF
Kinaya	2511533008	IF	Ghinada	2511533008	IF
Annisa	2511532024	IF	Haliya	2511532002	IF
Syasya	2511531018	IF	Kinaya	2511533008	IF
Haliya	2511532002	IF	Syasya	2511531018	IF

Pass 1: Annisa - 2511532024 - IF; Kinaya - 2511533008 - IF; Ghinada - 2511533008
 Pass 2: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Kinaya - 2511533008
 Pass 3: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Haliya - 2511532002
 Pass 4: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Haliya - 2511532002

Selection Sort

Gambar 2 output Selection sort

Sorting Mahasiswa

Nama:

NIM:

Prodi:

Tambah			Hapus		
Nama	NIM	Prodi	Nama	NIM	Prodi
Ghinada	2511533008	IF	Annisa	2511532024	IF
Kinaya	2511533008	IF	Ghinada	2511533008	IF
Annisa	2511532024	IF	Haliya	2511532002	IF
Syasya	2511531018	IF	Kinaya	2511533008	IF
Haliya	2511532002	IF	Syasya	2511531018	IF

Pass 1: Ghinada - 2511533008 - IF; Annisa - 2511532024 - IF; Kinaya - 2511533008
 Pass 2: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Haliya - 2511532002
 Pass 3: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Haliya - 2511532002
 Pass 4: Annisa - 2511532024 - IF; Ghinada - 2511533008 - IF; Haliya - 2511532002

Bubble Sort

Gambar 3 output Bubble sort